Crystal-structure generation for large-cell alloys: CE + SQS instead of MatterGen

csp / structure-generation stage

2026-05-28

Contents

1 Why MatterGen alone is not enough **1**

2 Pipeline at a glance **2**

3 Stage 1 – SQS first sweep (cheap, no model fitting) **2**

4 Stage 2 – enumeration of distinct orderings (small cells) **2**

5 Stage 3 – cluster expansion (the main engine) **3**

6 Stage 4 – AIRSS-style search for non-fixed parents **3**

7 Stage 5 – DiffCSP as a MatterGen replacement (optional) **3**

8 Choice matrix **4**

9 Integration with uvsib **4**

10 Sanity checks specific to this stage **4**

11 Implementation – SQS stage **5**
11.1 What the stage does, end-to-end 5
11.2 Files 5
11.3 Tunable parameters (experimentally relevant) 5
11.4 Short usage 6
11.4.1 Analysis – `workchains/sqs_analysis.py` 7
11.5 Status 8

1 Why MatterGen alone is not enough

The MatterGen-based CSP stage in uvsib is the default entry point for a newly queued material. The shipped pre-trained model is capped at roughly 20 atoms per unit cell. That cap is fine for small intermetallics and binary chalcogenides on the cathode side but it is broken before we even start for several anode targets:

- **Pyrochlore A2B2O7**, space group Fd-3m. The primitive rhombohedral cell already contains 2 formula units = **22 atoms** (4 A + 4 B + 14 O). The conventional cubic cell has 8 formula units = 88 atoms.
- **Perovskite SrIrO3** (Pnma 3C and P6_3/mmc 6H). Pnma primitive is 20 atoms and any cation substitution scheme needs supercells beyond that.
- **Rutile (Ir,Ru,Sn)O2 solid solutions** at non-trivial fractions need 2x or 3x supercells.

So the problem is not “extend MatterGen for alloys”. The problem is that **generative CSP is the wrong tool when the parent lattice is known**. For the anode-side screen the parent topology is fixed by experiment and the design axes are (i) which cation sits on which crystallographic site and (ii) at what composition. That is a configurational problem, not a structure-prediction problem.

The right tool for that is cluster expansion, supplemented by SQS and enumeration. This note describes how those slot into the existing uvsib pipeline and where MatterGen still belongs.

2 Pipeline at a glance

```
queued material
  |
  +-- parent topology UNKNOWN  --> MatterGen CSP                (<= 20 atoms)
  |                                 +- DiffCSP / AIRSS-MLIP     (larger cells)
  |
  +-- parent topology KNOWN    --> SQS supercells (icet/mcsqs)  (one per comp.)
                                   +- enumlib distinct orderings  (small cells)
                                   +- cluster expansion (icet)    (active loop)
                                            |
                                            v
                                     MLIP relaxation backend
                                     (MACE / MatterSim / UMA / uPET)
                                            |
                                            v
                                     convex hull on MLIP energies
                                            |
                                            v
                                     DFT validation of near-hull / low-eta candidates
                                            |
                                            v
                                     CHE Delta-G ladder (Sections 7-8 of the proposal)
```

The MatterGen branch is unchanged. The “parent known” branch is new and is what this document specifies.

3 Stage 1 – SQS first sweep (cheap, no model fitting)

Before fitting anything, generate **one** representative supercell per composition with Special Quasirandom Structures. SQS picks the supercell whose pair/triplet correlation functions best match those of the truly random alloy. It gives a defensible single-snapshot energy per (parent, composition) in minutes.

- Tool: icet’s `enumerate_structures` + `target_cluster_vectors`, or `mcsqs` from ATAT. icet is preferred because it shares ASE Atoms with the rest of uvsib.
- Output: one supercell per (parent, A-site composition, B-site composition).
- Cost: minutes per composition once the parent + supercell shape are fixed.
- Use it for: the **first** convex-hull sketch across the whole composition grid, before committing to a CE fit.

This is the analogue of “run one DFT per composition” but configurationally honest.

4 Stage 2 – enumeration of distinct orderings (small cells)

For small supercells we can enumerate **every** symmetrically distinct decoration and relax all of them. This catches the true ordered ground states that a CE will later have to reproduce; it is the ground-

truth check on the CE fit, not a substitute for it.

- Tools: `enumlib` (Gus Hart) directly, or pymatgen's `EnumerateStructureTransformation` which wraps it.
- Practical range: 1x-2x supercells, binary substitutions on one sublattice at fractions 1/4, 1/3, 1/2, 2/3, 3/4. For pyrochlore B-site Ru<->Ir this is tractable; for full A,B simultaneous substitution it explodes combinatorially and is not the right tool any more.
- Output: complete list of distinct orderings at chosen compositions, plus ordered ground-state energies.
- Use it for: validating the lowest CE-predicted orderings, and for getting the ordered ground state at "clean" stoichiometric points (1/4, 1/2, ...).

5 Stage 3 – cluster expansion (the main engine)

Cluster expansion fits the configurational energy of a fixed parent lattice as a sum of effective cluster interactions (ECIs) over occupation variables. Once fitted, evaluating an arbitrary decoration is essentially free, so the hull can be searched exhaustively or by Monte Carlo at any temperature.

- Tool: **icet** (https://icet.materialsmodeling.org). Pythonic, ASE-native, AiiDA-friendly. Alternative: CASM (more mature for finite-T thermodynamics and grand-canonical MC, more setup friction).
- Workflow:
  1. Choose parent lattice + sublattices to substitute (pyrochlore A 8c and B 6c; rutile M 6c; perovskite A 12c / B 6c).
  2. Generate initial training set: a few hundred random decorations across compositions, **MLIP-relaxed** (not DFT). MACE / MatterSim / UMA are already available as uvsib backends.
  3. Fit ECIs with LASSO or ARDR (icet supports both).
  4. Predict lowest-energy decorations per composition; if any look new, add them to the training set and refit. This is the **active loop**.
  5. Once the hull is stable under refit, DFT-validate only the points near the hull and the few low-overpotential outliers.
- Why not DFT-train the CE directly: with the MLIP backend in the loop we can afford hundreds of training points per parent at near-zero cost, which is what a trustworthy CE needs. DFT comes in at validation only.

This is the modern recipe; one-shot CE on a small DFT set is fragile.

6 Stage 4 – AIRSS-style search for non-fixed parents

CE assumes the parent topology does not change. That assumption breaks for:

- **Defective pyrochlores** (oxygen-deficient delta-phases, weberite-type distortions).
- **Reconstructed surfaces** of the same compositions.
- **New polymorphs** discovered along the substitution path.

For these cases the right tool is random structure search relaxed by the same MLIP stack. AIRSS (Pickard) is the simplest framework: generate sensible random structures with cell + symmetry constraints, relax them all, keep the unique low-energy ones. USPEX / CALYPSO are heavier evolutionary alternatives.

This branch is **optional** in the pipeline. Turn it on only when there is evidence the parent topology is not fixed (failed CE fit, large MLIP forces on the SQS snapshot, experimental indication of a distortion).

7 Stage 5 – DiffCSP as a MatterGen replacement (optional)

If we want to keep a generative CSP stage for the genuinely unknown-topology cases but lift the 20-atom cap, **DiffCSP / DiffCSP++** is the closest drop-in: diffusion-based crystal generation without

MatterGen's strict atom-count limit. Less mature than MatterGen but actively published. We do not need this for the anode screen; it is a future replacement for the MatterGen branch on the cathode side if its cap becomes a bottleneck there too.

8 Choice matrix

Parent	Composition axis	Right tool
unknown, small cell	–	MatterGen (current default)
unknown, > 20 atoms	–	DiffCSP / AIRSS + MLIP
known, single composition	–	direct relax, no generator
known, one sublattice substituted	continuous	**CE (icet) + SQS**
known, two sublattices substituted	continuous	**CE (icet)** (SQS for first sketch)
known, small supercell, all orderings wanted	discrete fractions	**enumlib**
known but distortion suspected	–	AIRSS-MLIP on top of CE

9 Integration with uvsib

The MLIP relaxation backend, the band-alignment workchain, the adsorbate sanity checks (`codes/files/adsorbates.py`) and the CHE ladder all stay unchanged. The new pieces are:

1. A `csp/icet/` module that wraps icet's CE-fit + sampling and exposes the same `(parent_template, composition_dict) -> [Atoms]` interface that the downstream relaxation expects.
2. A `csp/sqs/` thin wrapper around icet's SQS generator with the same interface.
3. A dispatcher in the structure-generation stage of `MainWorkChain` that selects MatterGen / DiffCSP / CE / SQS / enumeration based on the `parent_topology` and `n_atoms_estimated` flags of the queued material.
4. An optional `active_learning` toggle on the CE stage that triggers the refit loop until the predicted hull stops moving.

Dependencies to add:

```
pip install icet                    # CE + SQS + enumeration glue
pip install ase                     # already present
# enumlib binary if pymatgen's EnumerateStructureTransformation is used
# ATAT (mcsqs) only if we want a second SQS reference; optional
```

10 Sanity checks specific to this stage

- **CE validation**: hold out 10-20% of training structures, report MAE and RMSE of predicted vs MLIP-computed energies. A CE with MAE > 5 meV/atom is not trustworthy for hull work.
- **Hull stability under refit**: track the set of predicted ground-state structures across active-loop iterations; declare convergence when the set stops changing for two consecutive iterations.

- **MLIP vs DFT cross-check**: for each parent template, take 5-10 near-hull structures and DFT-recompute. If the MLIP systematically misranks by more than ~10 meV/atom, fall back to DFT for that parent or retrain the MLIP on the family.
- **Symmetry preservation**: after MLIP relaxation, confirm the parent space group via spglib (with a generous tolerance ~0.05 A). Loss of the parent symmetry is the signal to drop into the AIRSS branch.

11 Implementation – SQS stage

The SQS branch of stage 1 is implemented as the AiiDA plugin under `codes/sqs/` plus the entry-point script `codes/files/sqs.py`. The script now does both *generation* and a *quick MLIP relaxation pass* in the same job, so its output is a list of relaxed structures with energies attached – ready to be consumed by the convex-hull step directly. The CE branch (stage 3) and the AIRSS branch (stage 4) are still spec-only.

11.1 What the stage does, end-to-end

1. For each request, walk the Cartesian product of the composition grid.
2. For each composition: generate `n_per_comp` SQS supercells with icet (pure-composition endpoints short-circuit icet and decorate the parent directly so the hull endpoints are always present).
3. For each SQS supercell: optionally build slab(s) per requested Miller index, and optionally introduce surface oxygen vacancies at the requested concentrations.
4. Relax every generated structure with the MLIP backend selected via `--ML_model` (MACE / MatterSim / UMA / uPET, dispatched through `_calculators.make_calculator`) using `BFGSLineSearch`, `fmax=0.05 eV/A`, `max 200 steps`.
5. Drop unconverged relaxations. Store the relaxed MLIP energy on each surviving structure as `properties["predicted_energy"]` (set via `atoms.info` before serialising to a pymatgen `Structure` dict).
6. Write `output.json = {"structures": [...], "metadata": [...]}` with one metadata row per surviving structure.

11.2 Files

File	Role
`codes/files/sqs.py`	entry-point script copied to `aiida.py` and executed by the CalcJob
`codes/sqs/calculation.py`	`SQSCalculation` CalcJob – copies sqs.py + helper, builds CodeInfo
`codes/sqs/parser.py`	`SQSParser` – reads `output.json` into a `Dict` output node
`codes/sqs/workchain.py`	`SQSWorkChain` – `BaseRestartWorkChain` wrapper
`workflows/settings.py`	adds `sqs_files_path` so the CalcJob finds the script

11.3 Tunable parameters (experimentally relevant)

The input schema is documented in the `codes/files/sqs.py` module docstring; this is the user-facing summary.

Knob	Purpose	Typical values
`sublattices`	which Wyckoff sites carry which element pool	A: lanthanides + Bi/Y; B: Ru/Ir/Os
`composition_grid`	per-sublattice cation fractions to scan	1/4, 1/3, 1/2, 2/3, 3/4 (synthesisable)
`supercell` (int / list)	volume budget for the SQS search (in *primitive* cells of the parent, as icet sees it)	`[2,2,2]` or 8
`n_per_comp`	SQS samples per composition (different seeds)	1 for a first sweep, 3-5 for variance
`surfaces[].miller`	slab orientation	`[1,1,1]` for pyrochlore, `[1,1,0]` for rutile
`surfaces[].min_thickness`	slab thickness in A	10-15 A
`surfaces[].vacuum`	vacuum gap in A	15 A
`defects.oxygen_vacancy.concentrations`	fraction of surface O removed	0.0, 0.125, 0.25
`defects.oxygen_vacancy.n_per_conc`	random vacancy configs per fraction	2-5
`sqs.cutoffs` (technical)	icet pair, triplet cutoffs in A	`[7.0, 4.0]`
`sqs.n_steps` (technical)	MC steps per SQS	5000 default

Pure-composition endpoints (e.g. `{A: {Y: 1.0}, B: {Ru: 1.0}}`) short-circuit icet – icet rejects "no swaps possible" – and the parent supercell is decorated directly. This guarantees the convex-hull endpoints are always present in the output.

The downstream MLIP relax stage consumes the output structures; it dedups near-duplicates via spglib + pymatgen `StructureMatcher`, so the SQS stage is allowed to emit slightly redundant random-vacancy configurations.

11.4 Short usage

```python
import json
from ase.spacegroup import crystal
from pymatgen.io.ase import AseAtomsAdaptor

# 1. Build a parent template (pyrochlore Y2Ru2O7 here).
prim = crystal(
    symbols=['Y','Ru','O','O'],
    basis=[(0,0,0), (0.5,0.5,0.5),
           (0.375,0,0), (0.4375,0.125,0.125)],
    spacegroup=227, cellpar=[10.2, 10.2, 10.2, 90, 90, 90],
)
parent_dict = AseAtomsAdaptor().get_structure(prim).as_dict()

# 2. Describe the configurational degrees of freedom.
request = {
    "parent_label": "Y2Ru2O7-pyrochlore",
    "structure":    parent_dict,
    "sublattices": {
        "A": {"sites": "Y",  "species": ["Y", "Gd", "Bi"]},
        "B": {"sites": "Ru", "species": ["Ru", "Ir"]},
    },
    "composition_grid": {
```

```
        "A": [{"Y": 1.0},
              {"Y": 0.5, "Gd": 0.5},
              {"Bi": 0.5, "Y": 0.5}],
        "B": [{"Ru": 1.0},
              {"Ru": 0.5, "Ir": 0.5},
              {"Ir": 1.0}],
    },
    "supercell":  [2, 2, 2],
    "n_per_comp": 3,
    "sqs":        {"cutoffs": [7.0, 4.0], "n_steps": 5000},
    "surfaces":   [{"miller": [1,1,1], "min_thickness": 10.0, "vacuum": 15.0}],
    "defects":    {"oxygen_vacancy": {"concentrations": [0.0, 0.125, 0.25],
                                      "n_per_conc": 2}},
}

# 3a. Run standalone (for debugging):
with open("input_structures.json", "w") as f:
    json.dump([request], f)
# then:  python codes/files/sqs.py

# 3b. Or submit through the AiiDA workchain:
from aiida.plugins import WorkflowFactory
from aiida.orm import List, Dict, load_code, Str

builder = WorkflowFactory("sqs").get_builder()
builder.input_structures = List(list=[request])
builder.code = load_code("python@my_computer")
builder.job_info = Dict(dict={"job_type":   "sqs",
                              "ML_model":   "MACE",       # MACE / MatterSim / UMA / uPET
                              "model_path": "/path/to/checkpoint",
                              "device":     "cuda"})
builder.local_label = Str("Y2Ru2O7-pyro-screen")
```

The CalcJob writes `output.json` with the schema documented in the script's module docstring: a `structures` list of pymatgen `Structure.as_dict()` entries (each carrying its MLIP-relaxed energy under `properties["predicted_energy"]` in eV) and a `metadata` list with one entry per surviving structure (parent label, composition, n_parent_atoms, natoms, kind in {`bulk_sqs`, `slab`, `slab_0vac`}, miller, slab_thickness, vacuum, vacancy_frac, sqs_seed). The convex-hull step of `SQSWorkChain` consumes both lists directly – no separate relax stage is needed for SQS output.

Because the relax pass runs inline, the job_info dict must specify a real MLIP backend (`ML_model`, `model`/`model_path`, `device`, optionally `task_name` for UMA) – the placeholders in the example above are for illustration only.

11.4.1 Analysis – `workchains/sqs_analysis.py`

`SQSWorkChain.create_hull` calls three pure helpers and writes one combined `analysis` Dict output:

- `analyse_bulk(bulk_items, functional)` – one convex hull per `parent_label`. Elemental references come from `codes/files/{r2scan,gga_ggau}_entries.json` (selected by `functional`, defaults to `settings.DFT_FUNC`). Returns `eform_per_atom`, `ehull`, `energy_per_atom` and an `is_ground_state` flag per (parent, composition) after StructureMatcher dedup of equivalent SQS seeds.
- `analyse_surface(slab_items, bulk_results)` – surface energy `gamma = (E_slab - N * mu_bulk_per_atom) / (2 A)` per (parent, comp, miller). `mu_bulk_per_atom` is the lowest energy_per_atom from the bulk pass at the same composition; `area = |a x b|` of the slab cell. Reported in both eV/A2 and J/m2; entries with no bulk reference at that composition get `gamma = None` and a note.

- `analyse_ovac(slab_vac_items, slab_results, mu_O2)` – `dE_vac = (E_vac + 0.5 * n_vac * mu_O2 - E_pristine) / n_vac`. Pass `mu_O2` in eV per O2 molecule (MLIP-relaxed); without it the function falls back to the unreferenced `(E_vac - E_pristine) / n_vac` and tags the entry `reference="no O2 ref"`.

The O2 chemical potential `mu_O2` is computed automatically by the SQS CalcJob, *not* by the user:

- `codes/sqs/calculation.py` ships `codes/files/molecular_references/o2.vasp` into the remote calc folder alongside `aiida.py` and `_calculators.py`.
- After relaxing the SQS structures, `codes/files/sqs.py` `main()` calls `relax_molecular_reference(calc, "o2.vasp", "O2")` – same MLIP, same BFGSLineSearch settings, divides the relaxed cell energy by the four O2 molecules packed into o2.vasp – and writes the per-molecule value into `output.json` under `mu_O2` (and inside `molecular_refs` for any future H2O / CO2 / … references we add the same way).
- `SQSWorkChain.inspect_sqs` reads `output_dict["mu_O2"]` and stores it on the context. An explicit `mu_O2` input on the WorkChain (or `SQS.mu_O2` in `input.yaml`) overrides the auto-computed value when you want to pin a reference (e.g. for sensitivity tests or to use a beyond-MLIP DFT value).

Cost is ~0.5 s of MLIP inference per SQS run – cheaper than maintaining a per-(model, checkpoint) cache, and the value used for any given hull stays provenance-linked to the CalcJob output node that produced it.

11.5 Status

- **Stage 1 (SQS + inline MLIP relax)**: implemented in `codes/files/sqs.py` and the `codes/sqs/` AiiDA plugin. Generation end-to-end smoke-tested on a Y/Gd x Ru/Ir pyrochlore request (12 structures across 4 compositions: bulk + (111) slab + slab-with-O-vacancies per composition). The inline MLIP relax pass (BFGSLineSearch, fmax 0.05, max 200 steps) drops unconverged structures and stores the relaxed energy on each surviving structure under `properties["predicted_energy"]`.
- **Stage 1b (analysis)**: implemented in `workchains/sqs_analysis.py`, wired into `SQSWorkChain.create_hull`. Bulk hull + slab gamma + surface O-vacancy dE in one pass, returned as a single `analysis` Dict output. `mu_O2` is auto-computed inline by the CalcJob (relaxing `molecular_references/o2.vasp` with the same MLIP); explicit input still wins as an override. Smoke-tested on a synthetic 4-structure mini-dataset (numbers match the closed-form values for gamma and dE_vac to round-off).
- **Stage 2 (enumeration)**: not implemented; pymatgen's `EnumerateStructureTransformation` slot.
- **Stage 3 (cluster expansion, active loop)**: not implemented.
- **Stage 4 (AIRSS-MLIP fallback)**: not implemented.
- **Workchain dispatcher**: `MainWorkChain._construct_sqs_builder` is still a copy of the nanoparticles builder and does not yet pass the request schema described above. Update it to feed `List(list=[request, ...])` into `builder.input_structures` when wiring up the first real screen.
- **First user**: anode screen of the proposal (Section 8, OER pyrochlore).